Project: Service Mesh & Observability

Due: 04/18/2026 11:59 PM
Extended Deadline: 04/19/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Istio
- Task Overview
- Task

Prometheus
- Task Overview
- Task

Open Telemetry
- Task Overview
- Task

Wrap Up
- Conclusion

Prometheus (Task)

Heads Up — You're Switching Repositories For Tasks 2 & 3

Tasks 2 and 3 take place in a **separate GitHub Classroom repository** — not your usual one. Before getting started, clone this handout into your VM's **home directory**.

Before diving in, make sure you've reviewed the **Troubleshooting & Debugging Cloud Native Applications** primer if you haven't already — you'll need those debugging skills here. Find it on the course schedule.

Tasks 2 and 3 challenge you to take a broken codebase, get it running on a Kubernetes cluster, and wire up **Prometheus** and **OpenTelemetry** for full observability and instrumentation — then use what you see to track down and fix what's wrong.

At this point, we could put our application into production. We would be able to serve several clients, authorize requests, and manage our application fairly seamlessly. However, when our application is in production, we will run into a fairly big issue - visibility. Our application is spread throughout a number of different services and a number of external services. This makes it fairly hard to both debug and develop for full end to end testing.

What we want to be able to do is peer into the services, and tell what exactly our application is up to when it is deployed. This is known as **observability** in Cloud Native development, as our goal is to **observe** what is going on and make it entirely clear how to handle everything thrown our way.

As, at this point, our application is fairly involved, we have prepared detailed setup instructions for you to

complete. Setup your cluster following these instructions and the starter code **exactly** this time around, as otherwise we may be unable to grade your application.

Setup

Information

Link to the Handout:

```
https://classroom.github.com/a/Ukco6gu5
```

For this project, **please use the included source files instead of writing your own**. Unlike previous projects, where the main task was in **building** an application, the goal for both of these tasks will be to **diagnose** a (potentially) subtle error with the application using the logging tools provided. We will provide some high-level guidance to make sure that you are focusing your attention on the important sections, but to make sure that you have a reasonable solution you **must** debug the solutions provided, instead of writing your own code to solve the task.

Rename the folder from project-7-observability-<GITHUB_ID> to `broken-handout`.

Inside this renamed folder, please go to the folder titled `src`. Under here you will see the following folder structure:

```
handout/
    src/
        django/
        fastapi-p4/
        gateway/
        krakend/
        locusttest/
```

This is a variation of the solution to Project 3, using the scaled-up and modified versions of the Django database and the FastAPI version of the app we made in the last two projects.

As a general overview:

`django` contains the application code, alongside the `yaml` files necessary to build the application from scratch using a database-as-a-service system.
`fastapi-p4` is just the FastAPI server, circa P4.
`gateway` contains YAMLs for creating the Istio Gateway.
`krakend` contains the YAML and JSON files for your krakend config.
`locusttest` contains a loadtest which you will need for this task.

Information

Don't worry about reading the code in too much depth at this point; the rest of this section contains detailed instructions to setup your cluster and ensure that everything is up and running as we expect. We understand that this is a lot, but we do not expect you to run into any issues with this step. Please let us know if you do run into any issues. You can trudge ahead and not use the setup instructions provided, but it will make your life easier if you do.

Provision your cluster

At this point, these commands are probably well-known, but here is the command set to install and provision the P7 cluster.

First, we will login to our Azure shell:

```
az login --use-device-code
```

Then, we will create our project7 resource group and azure container registry (ACR):

```
az group create --name project7 --locatior
az acr create --resource-group project7 --
```

Then, we will login so that `docker` can connect to the ACR:

```
az acr login -n INSERTCOOLACRNAME
```

We will then provision and attach the ACR to the cluster:

```
az aks create --resource-group project7 --
az aks update -n project7cluster -g projec
```

Lastly, we will login to the cluster accordingly:

```
az aks get-credentials --resource-group pr
```

As we have done in past projects, you can check your connection by using `kubectl get nodes`. We expect that you will get something like the following:

```
NAME                                STATUS
aks-nodepool1-94542347-vmss000000   Ready
aks-nodepool1-94542347-vmss000001   Ready
```

Setup the Modernized Application Repository

We need to setup and run the following:

1. Our Istio Deployment.
2. Our KrakenD API Gateway.
3. Our FastAPI Deployment.
4. Our Django Deployment for Management.

Setting up the Istio

First, we will need to install the Gateway CRDs:

```
kubectl apply -f https://github.com/kuberr
```

Then we will install Istio's helm charts:

```
helm repo add istio https://istio-release.
helm repo update helm install istio-base i
 helm install istiod istio/istiod -n istio
```

You can check that this is valid through running:

```
kubectl describe gatewayclass
```

We expect to get output similar to the following:

```
Name:         istio
Namespace:
Labels:       <none>
```

```
Annotations:  <none>
API Version:  gateway.networking.k8s.io/v1
Kind:         GatewayClass
Metadata:
  Creation Timestamp:  2026-01-25T20:46:05
  Generation:          1
  Resource Version:    14447
  UID:                 53c4a8f9-e1de-4066-
Spec:
  Controller Name:  istio.io/gateway-contr
  Description:      The default Istio Gate
Status:
  Conditions:
    Last Transition Time:  2026-01-25T20:4
    Message:               Handled by Isti
    Observed Generation:   1
    Reason:                Accepted
    Status:                True
    Type:                  Accepted
Events:                    <none>


Name:         istio-remote
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  gateway.networking.k8s.io/v1
Kind:         GatewayClass
Metadata:
  Creation Timestamp:  2026-01-25T20:46:05
  Generation:          1
  Resource Version:    14449
  UID:                 13a005cd-7291-42b8-
Spec:
  Controller Name:  istio.io/unmanaged-gat
  Description:      Remote to this cluster
Status:
  Conditions:
    Last Transition Time:  2026-01-25T20:4
    Message:               Handled by Isti
    Observed Generation:   1
    Reason:                Accepted
    Status:                True
    Type:                  Accepted
Events:                    <none>
```

You can finally then install the resources under the `gateway` folder using:

```
kubectl apply -f gateway/
```

Setting up the KrakenD Configuration

Under the `krakend` folder, we first need to create our **configmap** for our system. Make sure you are inside of the `krakend` folder before running:

```
kubectl create configmap krakend-cfg --fro
```

For your convenience's sake, we have provided a private key file for you to use. It is not necessary to use this file, and it certainly is not best practice, but it should simplify some of the steps here. To add the secret run:

```
kubectl create secret generic krakendjwt -
```

Once both of these files are added, you should be able to deploy the KrakenD pods using:

```
kubectl apply -f krakend-deployment.yaml
```

We can check that everything is running as expected by first checking that the deployment logs are running using:

```
kubectl logs deploy/krakend-deploy
```

You should get something like the following:

```
Parsing configuration file: /etc/krakend/k
 2024/04/07 - 04:34:16.427 ▶ DEBUG {"@tim
 2024/04/07 - 04:34:16.427 ▶ INFO {"@time
 2024/04/07 - 04:34:16.428 ▶ INFO {"@time
 2024/04/07 - 04:34:16.428 ▶ DEBUG {"@tim
 2024/04/07 - 04:34:16.428 ▶ INFO {"@time
 2024/04/07 - 04:34:16.428 ▶ DEBUG {"@tim
 2024/04/07 - 04:34:16.428 ▶ DEBUG {"@tim
 2024/04/07 - 04:34:16.428 ▶ DEBUG {"@tim
 2024/04/07 - 04:34:16.428 ▶ DEBUG {"@tim
 2024/04/07 - 04:34:16.428 ▶ DEBUG {"@tim
 2024/04/07 - 04:34:16.429 ▶ INFO {"@time
```

```
2024/04/07 - 04:34:16.429 ▶ INFO {"@time
2024/04/07 - 04:34:21.428 ▶ DEBUG {"@tim
```

With our KrakenD Pods setup, let's attach our LLM service and our Django service as well, and then confirm that everything is working as expected.

Applying our FastAPI LLMService.

First, we will need to build our FastAPI service for our architecture. We will do this through the docker `buildx` command for our purposes. This is something that you are already familiar with.

```
docker buildx build --push --tag INSERTCO(
```

Information

This should work for your application, but if it does not your AKS might use a different architecture than us. You can look for the architecture with:

```
kubectl describe node | grep "Architecture
```

For our application, we get the following:

```
Architecture:                amd64
Architecture:                amd64
```

If you get another value here, you should be able to change the platform from `linux/amd64` to `linux/INSERTARCH`. This is not meant to take up that much of your time - please let us know if you run into any errors here.

After pushing the pod, you will need to change the image address in the associated kubernetes YAML files, after which you should be able to deploy your LLM service through:

```
kubectl apply -f k8s
```

Applying our Django Service.

Last, but certainly not least, we need to deploy our Django service. For this task, we will use a managed MySQL solution called MariaDB for our deployment, as

it will be a bit more replicable and a bit easier to manage.

First, using either the provided keys or your own, create the key secret for our Django application:

```
kubectl create secret generic loginkeys --
```

Then, we will need to install MariaDB's operator file, which will allow for easy provisioning and configuration of our database:

```
helm repo add mariadb-operator https://mar
helm install mariadb-operator mariadb-oper
helm install mariadb-operator-crds mariadb
```

After installing the Helm chart, run the following to provision our database:

```
kubectl apply -f mariak8s/mariadb-config.y
kubectl apply -f mariak8s/mariadb-secret.y
kubectl apply -f mariak8s/mariadb-pod.yaml
kubectl apply -f mariak8s/mariadb-db.yaml
```

By default, it is a read-replicated database, but instead of provisioning it manually like we did in the last project, MariaDB will manage the issues of scaling, replication, and fault-tolerance for us, making it a bit more resilient than our original database.

Feel free to read the manifests if you are interested, but there really isn't much there that is not pulled from the example manifests provided by MariaDB - we just configure the database so that it points to the correct system, is replicated, contains the database `login`, and has the password `password` for our `root` user.

Using the following command you can verify that the database setup is working:

```
kubectl run mysql-client --image=mysql:8.0
```

After applying these manifests, you can use a similar command to build your Docker image for the Django system:

```
docker buildx build --push --tag INSERTCO(
```

While we don't need to put a specific platform on this image, you may want to if you run into issues getting the container to run.

After building the image, you can run the following to make the deployment and service come up, after editing the manifests to point to your images and ACR appropriately.

```
kubectl apply -f k8s
```

Setup Check

As we did go through a monumental number of setup steps, let's just make sure that we are on the same page when it comes to your application functioning as expected. To do so, we will use `curl` like we did in P3, though in the coming steps we will move on to better systems for checking that our system is working.

First, get the `Gateway` IP address with:

```
kubectl get gateway
```

You should get an IP address like the following:

```
NAME      CLASS   ADDRESS        PROGRAMMI
gateway   istio   20.84.16.127   True
```

We can then first curl into our login service to confirm that it is functioning as expected:

```
curl -H 'Content-Type: application/json' -
```

We can confirm that our access token lets us use our API with:

```
curl -D - -H  "Authorization: BEARER INSEI
```

You should get some output like the following:

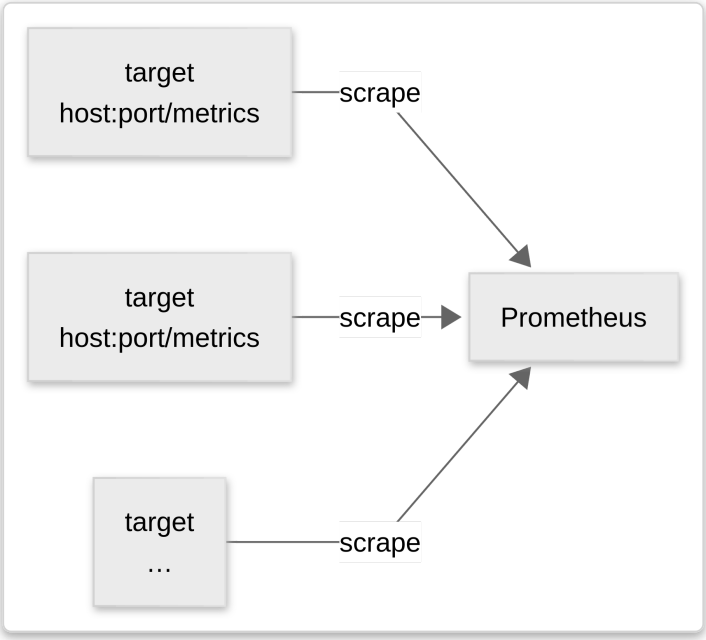
```
HTTP/1.1 200 OK
content-type: application/json; charset=ut
x-krakend: Version 2.5.1
```

```
x-krakend-completed: true
date: Mon, 26 Jan 2026 14:27:05 GMT
content-length: 821
x-envoy-upstream-service-time: 3
server: istio-envoy

{"choices":[{"finish_reason":"length","inc
```

With this done, remember that you implemented 99% of this application across the entire semester. You build the application, iterated across multiple designs, and have gotten quite drastic performance improvements through the use of better tooling. While it is probably over-complicated for the task at hand, it is scalable and allows for independent operation between those writing the login service and those writing the API service - the benefits of being Cloud Native!

Subtask 1: Adding Prometheus



With that said, you can probably imagine the headache that it would take to check this application for any subtle bugs and ensure that every service is running as expected. There are a **lot** of moving parts, and it is easy to see that any particular request could fail at any of the following places:

1. Our Django code.
2. The MariaDB database.
3. Our FastAPI code.
4. The API Gateway (KrakenD).

5. Our Gateway Istio Pod.

In practice, managing all of this would be crazy, and require a ton of work to do manually. While we could have you implement a simple system for doing so, we will instead focus on demonstrating how you would add observability to this application directly.

As you may remember from the conceptual content, **observability** is a long word that literally just indicates logging if our system is working as expected or not. We don't know with any certainty what could fail in the sheer number of pods and other applications we have installed, and so we need systems that can both enable real-time analytics and post-runtime analytics. For this task, we will focus on enabling real-time analytics through **Prometheus**, a common time-series database used mostly for providing real-time metrics for your Cloud Native applications. If you did Project 4 Task 4, you saw a little bit of Prometheus through Istio, which can handle some of the management for you. For our purposes, we will instead focus on showing how to manually connect our application to Prometheus through the **Prometheus Operator**.

> **Information**
>
> We will note that all of this is just one of many ways to handle Prometheus integration. Prometheus is a fairly powerful tool all things considered, so we will focus on building familiarity with the tool, and the high-level pros and cons of using it, for this exercise.

To install Prometheus, we will start by adding the appropriate Helm repo:

```
helm repo add prometheus-community https:/
```

Then we will run the following to install the chart:

```
helm repo update
helm install prometheus prometheus-commun
```

That's about it for installation. At it's core, Prometheus is fairly simple to setup, and we can already see some cool metrics and get a better idea of what we can do before we discuss the meat of what we want you to do.

Subtask 2: Metrics as Glorified Time Series

Now that you have installed Prometheus, the first thing we would like you to start running in the background is the provided locust test. To do so, run the following:

```
AUTH_TOK=$(curl -H 'Content-Type: applicat
poetry install
poetry run locust -f locusttest/locustfile
```

For the sake of providing an accurate speed comparison, we have relaxed some of the rate-limiting parameters and throw 3 users at a single pod. Prometheus focuses on providing "real-time" metrics, as opposed to logs of pods and other associated information. It's best for learning the current state of the cluster, from a more "what resources are being used" perspective, compared to anything else.

The first thing we will take a look at is Prometheus's "default dashboard", which is really more of a fancy SQL-Workbench-like website, built for those who know what queries they can get with Prometheus and just want to get a quick glance at some custom data. In order to access the website, you will need to forward the port on the pod to your computer using:

```
kubectl port-forward -n prometheus prometh
```

If you are using the VSCode Remote-SSH extension, this is sufficient. However, if you are SSH-ing directly to your laptop, you will also need to forward the port from the remote computer to your local development environment using something like [the methods described in this article](#).

Either way, once you boot up the dashboard, you will see that it's rather stark:

Figure: Blank Dashboard

By default, Prometheus will take you to its "Graph" dashboard, which will let you play around with the metrics it gets for free from Kubernetes. This contains everything necessary to query Prometheus directly. Unlike SQL and other relational query languages, PromQL is much simpler, and is more used to get time series to build dashboards directly.

While the query language is easier, we do want to mention that the associated data is anything but easier. This is further complicated by the fact that, as we mention in the conceptual material, Prometheus is pull-based, meaning if a metric source exists it will try to pull from it. If you click on the "Status" section of your repository, and go to the "TSDB Status" section, you will see that we have on the order of tens of thousands of series. After running the project and a load test for a day, we get around 45K separate time series - each of which may contain data relevant for your application.

Given this is literally too much data for you to work with, let's re-think this approach from the top-down as opposed to the bottom-up. In order to get the relevant metrics, Prometheus needs to get the information from a source - this could be something as "simple" as Kubernetes or as complex as on of our pods.

If you go to the "Targets" section under metrics instead, you can see all of the metric source for our page in a fairly easy-to-read format. In the Labels section, there is a label with the name "job". This points out the source of the metrics given.

After some Googling, it becomes clear that there are the following metric sources:

1. Kubernetes itself. This provides the vast majority of metrics, for every pod, node, and literally everything else on the cluster.

2. KrakenD, our API Gateway. Our KrakenD.json file is providing tons of logs to Prometheus as we enabled the logging during P2. While before this was mostly a deluge of information, we will be able to get a better idea of how good our system is this way.
3. The Prometheus Alert Manager, which would allow us to notify people when metrics go beyond expected values.

These provide a large portion of fairly relevant data. While we could use `kubectl top` and `kubectl logs` to get a running set of information, Prometheus has already done the hard work of processing it and storing it for us. While all of this data is currently stored in memory, you could choose to persist it as well. However, for this project, we will instead focus on what is **missing**, and make sure we try to include relevant metrics as well.

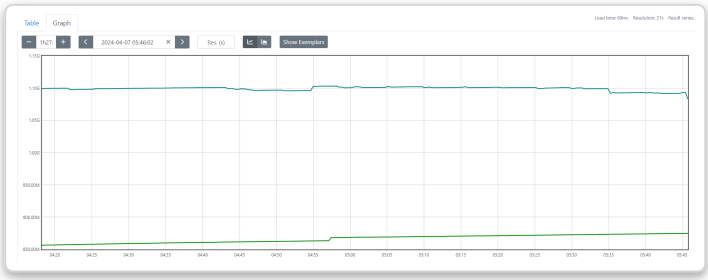
Specifically, if we think about our system wholistically, we have some fairly major blind-spots:

We have no metrics that are specific to our Django and FastAPI services. While it is nice to know that KrakenD has a request metric set, we cannot use KrakenD as a proxy for the real requests accepted / denied by our service, as KrakenD employs retries and other behaviors to ensure that the system works end to end.
We have no metrics for our externally-facing gateway, Istio Gateway Fabric. If Istio malfunctions in some way, we would really like to know of it as soon as possible so we can course-correct.
We have no metrics for our database. While we have some trust that MariaDB will not break down, we would like to get an idea of if our database has hit maximum throughput and/or what the load on it is like from a database POV as opposed to just resource utilization.

Luckily for us, we will fix all of these issues in the next task. For now, let's focus on understanding a couple of key metrics we have, as it has probably been about 5-10 minutes since you started the load test.

Going back to the query view, let's start with a metric that is likely to be on your mind - the number of authenticated requests. To get a metric from the PromQL workbench, we simply need to put its name as

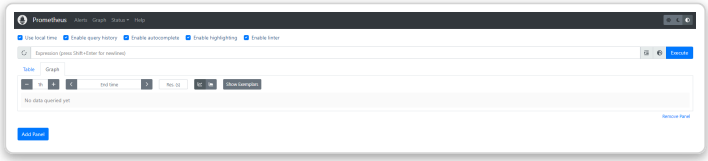
a query. In this case, the name is `authenticated_user_requests`:



As you can probably expect, there's not much to see here. While your y-axis is likely very different from ours, as ours comes after running the cluster for 10-12 hours, you likely see both some time with 0 requests, when you did not run the load-test, alongside some time with a the number of requests linearly increasing like the diagram we have here, corresponding to when our load test started. While we could use our Locust logs, these logs are cluster-wide, and allow us to get a really good idea of what the traffic will look like in-production.

If you can't see the change, you are not out of luck. On the GUI, you can change the "15m" selector to an hour or more to see the rest of the data. By default, Prometheus stores all of the data it can wherever it can, only deleting old data when necessary.

Once you have confirmed that your load-test metrics are working as expected, we want you to look at another metric, one a bit more concerning - the total memory usage of our cluster. If you put in the metric name - `node_memory_Active_bytes` into the PromQL browser, you will see the following:



You may not see it yet, but the total memory utilization for our cluster is slowly increasing, at a fairly slow rate of less than a Mb per second. This is because somewhere in this model deployment we have added a memory leak, where our program is slowly but surely taking up more and more memory. While in some situations this is fine, we will tell you that this memory leak will lead to the program crashing after a few weeks of user-interaction. As we want our application to always be online, and know that our application does work, this is

not an error that we can assess through better unit testing, as it is solely based on performance at scale.

Now that we have identified an issue with our system (a potential memory leak), we would like to localize the error somewhat before you go and try to fix the system. We would like to understand two separate but important issues:

1. What percentage of the node's memory is this leak taking up, as a function of time?
2. What is causing the leak? Is it something in a Docker container that we made, or one or more of the external systems we are using?

For the first question, PromQL provides a nice syntax for dividing time series values, which will allow us to compute the percentage of memory utilized by our program. In our case, the query will look something like the following:

```
node_memory_Active_bytes/node_memory_MemTo
```

Here, we are dividing our active memory utilization by our total memory utilization, and then re-scaling so we have percentages. If you run this, you will see that the increase is around 0.05% per 30 minutes or so. Again, while this does not sound that bad, it would potentially use up the entire cluster's memory in about 2 months, assuming we kept everything at only 1 pod. If you can guarantee that someone will monitor the system that closely for that long, it's probably fine. However, if the system is mission-critical or needs to be scaled up, you could run into a larger issue that could crash within days depending on the number of additional FastAPI pods we run.

For the second question, we will look at a different metric:

```
node_namespace_pod_container:container_mem
```

Here, everything before the colon just denotes that we are working with a metric that is a per-container metric. If you put in this query as-is in your system, you will see around 37 different time series, each corresponding to the memory usage of a particular container.

At this point, look through this view of the data for a bit, trying to see what the rogue container is before we discuss what the container is exactly. If you use the load test provided, it should be fairly clear what pod is growing.

START ACTIVITY

Complete the Activity to Unlock the Next Section

As you have correctly identified - our llmserver has a memory leak. We can take a look at it directly with a query that focuses on the pod itself:

```
node_namespace_pod_container:container_mer
```

Here, the curly-braces allow us to select **a particular series based on additional information, like the pod name**. The tilde (~) here is just how PromQL specifies that we want to match a series with a regular expression instead of the exact name, allowing us to see all containers related to our llmserver deployment.

If we want to get the total memory utilized over all pods, PromQL has a handy "sum" function, which we can use through the following:

```
sum(node_namespace_pod_container:container
```

Now that you have located the issue, you will need to debug and solve the issue accordingly. This may sound daunting, but here are a few hints:

1. Fixing the bug itself involves editing or removing at most 3-4 lines of code.
2. You do not need to worry about preserving what the line(s) is/are trying to do. You just need to remove the offending code for this task. So long as the memory leak doesn't exist, and your system still works, this subtask will be a success.
3. You do not need to worry about looking at Prometheus logs to find further issues. This is the only issue we intend you to fix.
4. When replacing the pod, you may run into an error where the pod fails for an unexpected reason. If this

happens, simply delete and re-create the deployment instead.

Validation

To check your progress, you can re-run the load-test for 10 minutes and check to see if the issue has been resolved. Don't skip the load test re-run. Wait until you see stable memory on Prometheus (around 5 minutes), then stop the test and submit your work. During submission, we will run this load-test as well, and then check your memory utilization to make sure that the memory is not slowly increasing at the rate it was doing so before. It is important to note that we do expect **some** memory growth as the pod runs, as llama-cpp-python has a cache that it populates for this purpose, but the total memory utilization should not increase as fast as it did previously.

Do NOT modify the loadtest ahead of submitting.

To submit for this task, use the following command. Make sure you are under the root directory of the repository, and that your loadtest file under `src/locusttest/locusttest` is not modified.

We are reusing the submitter from task1, ignore the submitter at ./broken-handout/submitter.

```
./submitter -t task2
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform